

# AI ASSISTED CODING ASS – 19.1

2303A51406

B-27

## Task 1 – Python to Java Conversion

**Task:** Translate a simple Python class into Java using AI assistance.

### Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

### Expected Output:

- Equivalent working Java class translated from Python.

### Python Code:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

s = Student("John", 20)
s.display()
```

### Output:

```
Name: John, Age: 20
```

## Prompt:

Convert this Python class into an equivalent Java class.

Requirements:

- Keep same logic and structure
- Convert `__init__` to constructor
- Use proper Java data types
- Include a main method
- Ensure the output is the same

## Java Code:

```
class StudentRecord {
    private String name;
    private int age;

    public StudentRecord(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        StudentRecord s = new StudentRecord(name: "John", age: 20);
        s.display();
    }
}
```

## Output:

```
Name: John, Age: 20
```

## Task 2 – C++ to Python Function Conversion

**Task:** Convert a C++ function for factorial calculation into Python.

### Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

### Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

### C++ Code:

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int num = 5;
    cout << factorial(num);
    return 0;
}
```

### Output:

120

### Prompt:

Convert this C++ recursive factorial function into Python.

Requirements:

- Keep the same recursive logic
- Use Pythonic syntax
- Handle base cases properly
- Include a test call to the function with an example input

### Python Code:

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
# Test call  
print(factorial(5))
```

### Output:

120

## Task 3 – SQL to Pandas Query

**Task:** Translate a SQL query into a Pandas DataFrame operation in Python.

### Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

### Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

## Prompt:

Convert the following SQL query into an equivalent Pandas DataFrame operation in Python.

Requirements:

- Use proper Pandas syntax
- Assume the data is already loaded into a DataFrame called 'df'
- Ensure filtering and column selection are correct
- Provide complete Python code including sample DataFrame and output

SQL Query:

**SELECT name, salary FROM employees WHERE salary > 50000;**

## CODE:

```
import pandas as pd
# Sample DataFrame
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 30000]
}
df = pd.DataFrame(data)
# Equivalent Pandas operation
result = df[df['salary'] > 50000][['name', 'salary']]
print(result)
```

## Output:

```
#Output:
#      name  salary
#0   Alice   60000
#2  Charlie   70000
```

## Task 4 – Real-Time Application: Multi-Language Snippet Converter

**Scenario:** Students will choose a real-time use case where they need code in multiple languages.

### Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

### Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

### Expected Output:

- Working equivalent code snippets in at least two different programming languages

### Prompt:

Convert the following Python web scraping code into equivalent JavaScript code for browser automation.

Requirements:

- Maintain the same functionality
- Use appropriate JavaScript libraries (like axios, fetch, or puppeteer)
- Ensure the output is similar
- Provide complete working code

### Python Code:

```
import requests

from bs4 import BeautifulSoup

url = "https://books.toscrape.com"

response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

titles = soup.find_all("h1")

for title in titles:
    print(title.text)
```

## CODE:

```
const axios = require("axios");
const cheerio = require("cheerio");

const url = "https://books.toscrape.com";

axios.get(url)
  .then(response => {
    const $ = cheerio.load(response.data);
    $("h1").each((i, el) => {
      console.log($(el).text());
    });
  })
  .catch(error => {
    console.error(error);
  });
```

## Output:

```
C:\Users\HARSHAVARDHAN\Desktop\AI Assitant Coding>node scrpit.js
All products
```